

SYSTEM REPORT

Equipment Health Monitoring & Alerting Application

Session-aware monitoring, health evaluation, incident alerting, and a live web dashboard for a third-party equipment management application.

Report date: 23 August 2026

Target application: jwintell.com (Equipment Management → Device Information)

Prepared for: Project owner / stakeholder review

Status: Phases 1–5 implemented and functionally verified live against the real target site; Phase 6 (hardening) and full Postgres/SMTP verification pending — see §8.

Table of Contents

1. [Executive Summary](#)
2. [System Architecture](#)
3. [End-to-End Workflow](#)
4. [Framework & Technology Choices — Justification](#)
5. [Constraints & Current Limitations](#)
6. [Feature Walkthrough \(Live Screenshots\)](#)
7. [Testing & Live Verification Summary](#)
8. [Pending Features / Roadmap](#)
9. [Appendix A — Tool & Technology Reference](#)
10. [Appendix B — Project File Structure](#)
11. [Appendix C — State Vocabulary Glossary](#)

1. Executive Summary

This application is a lightweight monitoring and alerting layer sitting in front of an existing, authenticated, third-party equipment-management web application (jwintell.com). It does not replicate that application. It periodically retrieves equipment status from its Equipment Management → Device Information section, converts the target's raw fields into a canonical health state via configurable rules, detects state changes exactly once per transition (never once per poll), stores full history, and notifies subscribers by email — with a live web dashboard for humans to watch the whole fleet.

The two requirements treated as non-negotiable throughout the build, per the original specification, were: **authentication must be reused, not repeated** (no login on every poll cycle, CAPTCHA handled only by a human), and **a monitoring/technical failure must never be mistaken for an equipment fault** (target unreachable, session expired, or extraction error are distinct technical states, never converted into a fake HEALTHY/OFFLINE/MALFUNCTION reading).

Phase status

Phase	Status	Notes
1 — Target application discovery	Done	Confirmed live: login flow, session markers, and — critically — the underlying JSON list API the target's own UI uses, discovered via live network inspection rather than assumed.
2 — Monitoring proof of concept	Done	Session reuse across repeated polls proven live (no re-login on repeat runs).
3 — Database + state engine	Implemented, live-verified	Against SQLite only — no Postgres/Docker available in the build environment (see §5, §8).
4 — Alerting	Implemented, live-verified	Incident-triggered notifications confirmed firing during live runs; no real SMTP send exercised (no credentials available).
5 — Web dashboard	Implemented, live-verified	Real uvicorn process, hit with real HTTP requests; screenshots in §6 are captured from this live run, not mockups.
6 — Production hardening	Not started	See §8.

What "live-verified" means in this report: every screenshot in §6 and every metric quoted in §7 was captured from an actual running process talking to the real target site during this build — not a mockup, not a simulated response. Where something has *not* been exercised for real (Postgres, SMTP), this report says so explicitly rather than presenting it as done.

2. System Architecture

The system is deliberately layered so that if the target site's markup or API changes, only the target-facing layer needs to change — nothing downstream of it should need to know or care where equipment data came from.

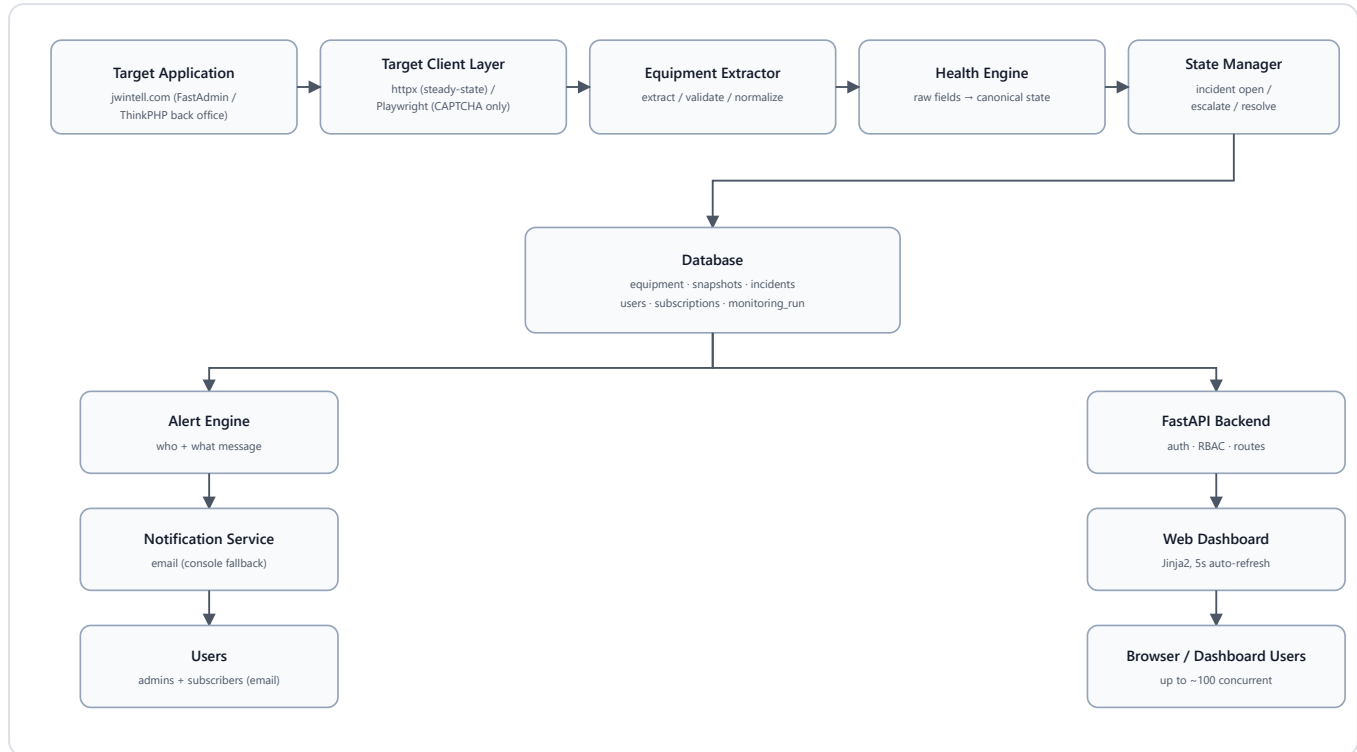


Figure 1 — Component architecture. The Database is the single integration point between the monitoring worker (left/top path) and the web dashboard (right path) — they never talk to each other directly.

Why this shape

- **Isolation of target-specific code.** Only `target_client.py`, `lightweight_client.py`, `mapping.py`, and `selectors.py` know anything about jwintell.com's actual markup/API. Everything from the Extractor onward works with normalized data.
- **Health is computed, not copied.** The target's own `status` field is never treated as the application's health verdict — the Health Engine applies independent, configurable rules (§4) to arrive at one of five canonical states.
- **One shared database, two independent processes.** The monitoring worker and the web dashboard are separate OS processes that never call each other; they only ever meet at the database. This matches the spec's own scale model (§29 of the original brief): one monitoring session, one dataset, many dashboard viewers.

3. End-to-End Workflow

Figure 2 shows one full monitoring poll cycle exactly as implemented in `monitoring/worker.py` — including the session-validity branch (which is where the CAPTCHA-driven, human-in-the-loop authentication happens) and the per-equipment state-transition branch (which is where the "one alert, not four" rule is enforced).

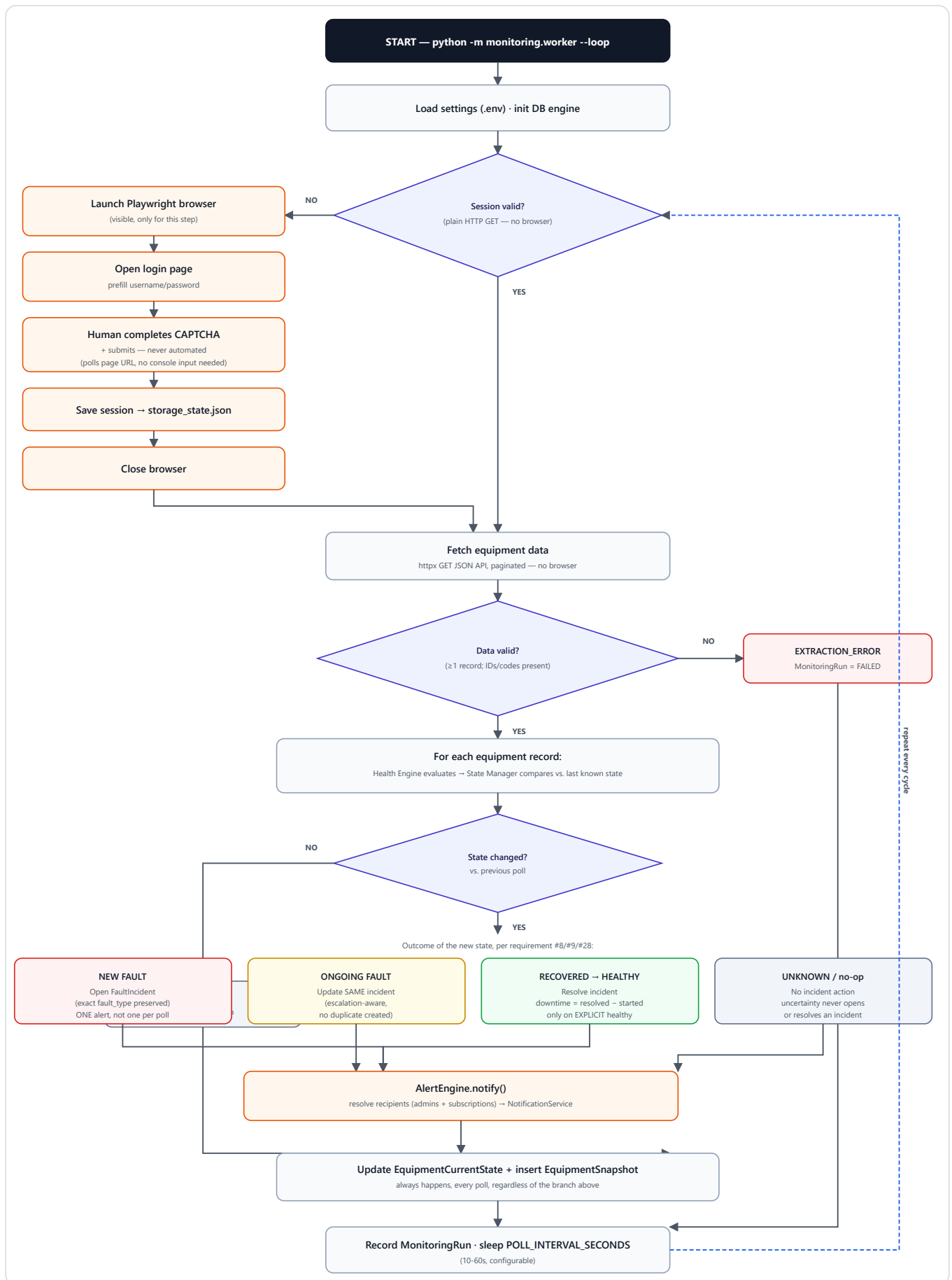


Figure 2 — One monitoring poll cycle, including the session-validity/CAPTCHA branch and the four possible outcomes of a state-transition check.

Narrative walkthrough

1. **Startup.** Settings load from `.env` ; the database engine initializes (tables created if absent).
2. **Session check.** `LightweightTargetClient.is_session_valid()` issues one plain HTTP GET to the target's authenticated home page and inspects whether it was redirected to the login page — no browser involved.
3. **Authentication (only when needed).** If invalid/expired, `_reauthenticate()` is the only place a Playwright browser is launched. Credentials are prefilled; the CAPTCHA is left entirely to a human. The resulting session cookies are persisted to `storage_state.json` , and the browser is closed immediately — it does not stay open for polling.
4. **Extraction.** Equipment data is fetched via the target's own JSON list API (discovered during Phase 1, preferred over DOM scraping per the original requirement to use a structured request where one exists), with a DOM-scrape fallback available in the Playwright-based client if the API ever breaks structurally.
5. **Validation.** An empty or malformed result set is treated as a validation failure, never as "zero machines" — this is the safeguard against a broken page or a failed login silently producing a fleet-wide false OFFLINE reading.
6. **Health evaluation.** Each record is run through the configurable rule table in `config/health_rules.yaml` to produce one of five canonical states.
7. **State comparison and incident handling.** The new state is compared to the last known state. A fault only ever opens ONE incident for its entire duration; ongoing faults update that same incident; recovery is only recognized on an explicit HEALTHY observation; an `UNKNOWN` reading — from a missing or unrecognized field — takes no incident action in either direction.
8. **Alerting.** Any incident opened, escalated, or resolved this cycle triggers `AlertEngine.notify()` , which resolves recipients and sends via `NotificationService` .
9. **Persistence.** A snapshot and the current-state row are written for every equipment record on every cycle, regardless of whether anything changed — this is what powers the dashboard's history timeline.
10. **Bookkeeping and wait.** A `MonitoringRun` audit row records the cycle's outcome; the worker sleeps for the configured interval and repeats.

Web dashboard request flow

The dashboard is a separate, independent process. On each page request: a signed session cookie identifies the user (or redirects to `/login`); a role check gates admin-only pages; the route reads directly from the same database the worker writes to; a Jinja2 template renders the response. The fleet-overview, active-faults, and equipment-detail pages carry a 5-second `<meta http-equiv="refresh">` so an open browser tab reflects new polls without a manual reload.

4. Framework & Technology Choices — Justification

4.1 Playwright, but only for the CAPTCHA step

Playwright was the correct starting point because the target application requires an interactive login with an image CAPTCHA — no pure-HTTP client can complete that step, and per the original requirement, it must never be automated. However, once Phase 1 discovery revealed that the actual equipment data comes from a plain JSON endpoint (not server-rendered HTML requiring JS execution), keeping a full browser running for every subsequent poll stopped being justified. The application was refactored mid-build (see the git history) to a hybrid: Playwright is retained exclusively for the login/CAPTCHA flow and as a DOM-scrape fallback; `httpx` handles all steady-state polling. This was verified live — zero Playwright-owned Chromium processes existed across multiple full poll cycles once the change was made, versus one long-lived visible browser window before it.

4.2 FastAPI for the backend

Chosen over Flask/Django for its native `async` support (consistent with the rest of the codebase), automatic request validation via Python type hints, and built-in OpenAPI docs generation — useful even though the dashboard is server-rendered, since the JSON endpoints (`/api/monitoring/status`) are self-documenting for free. It is also explicitly the framework named in the original technology brief.

4.3 Server-rendered Jinja2, not Next.js

The original brief named React/Next.js. Node/npm were confirmed available in the build environment, but a Next.js frontend means a second application, a separate build pipeline, and a second dev server to run and keep in sync with the API — substantially more surface area than this deployment's scale (1–100 internal users) justifies. This was raised explicitly with the project owner before building, who confirmed the server-rendered approach. The route layer (`backend/api/*.py`) still returns structured data cleanly enough that a Next.js frontend could be layered on top later without touching `services/` or `db/`.

4.4 SQLAlchemy over PostgreSQL, with SQLite as a portable test/demo target

SQLAlchemy's ORM gives declarative models with real relationships and constraints, and — deliberately — the models in this project use only portable column types (no JSONB, no native ENUM) so the identical schema runs against both PostgreSQL (the specified production target) and SQLite. This was not a design accident: it made 70 unit/integration tests possible without any external database dependency, and made a full live demonstration of the worker + dashboard possible in an environment with no Docker/Postgres installed. §5 covers why this portability is a development convenience, not a production substitute.

4.5 Configurable health rules (YAML), not hardcoded logic

The mapping from raw target fields (`fault_status_text` , `online_status_text` , ...) to canonical health states lives in `config/health_rules.yaml` , not in code. This directly satisfies the requirement that new fault vocabulary must be addable without a code change, and keeps the actual rule engine (`services/health_engine.py`) a thin, well-tested evaluator rather than a place where business logic and mechanism are entangled.

4.6 Stdlib-only authentication for the dashboard

Rather than adding `passlib` / `itsdangerous` / a full auth framework, password hashing uses PBKDF2-HMAC-SHA256 via Python's built-in `hashlib` , and session cookies are signed with `hmac` — both well-understood, dependency-free primitives appropriate for this application's actual threat model and scale. This keeps the dependency surface smaller without giving up cryptographic soundness.

4.7 Why not APScheduler

The original brief suggested APScheduler for polling. The implemented loop (`asyncio.sleep` inside `while True`) is simpler and sufficient for a single fixed interval; APScheduler's value (cron-style multiple schedules, misfire handling, job persistence) isn't needed yet at one monitoring profile with one interval. This is flagged in §8 as worth revisiting if multiple monitoring profiles/schedules are added later.

5. Constraints & Current Limitations

5.1 Inherent to the target application

CAPTCHA requires a human, by design. This is not a limitation to be engineered around — the original requirement explicitly forbids automating it. Any deployment must have a person available to complete login the first time and whenever a session genuinely expires.

- **Single monitoring account/profile.** The current implementation supports one target-application login. Multiple target accounts (spec §29's "Monitoring Profile A / B") would need a profile abstraction not yet built.
- **Fault vocabulary is unconfirmed.** No device in the connected account has reported a non-"Normal" `fault_type` or `material_shortage_status` during this build, so the rule table in `health_rules.yaml` has not been validated against a real-world fault string beyond the illustrative examples in the original spec. The dashboard screenshots in §6 use clearly-labeled synthetic rows to demonstrate the Warning/Malfunction badges for this reason.
- **DOM-scrape fallback is unverified in practice.** It exists in `target_client.py` for resilience if the JSON API ever breaks structurally, but has never actually been exercised against a real failure — the JSON API path has been reliable throughout.

5.2 Data layer

Real PostgreSQL has never been used. Every DB-touching component targets PostgreSQL and is written against the real `postgresql+psycopg2` driver, but this build environment has neither Docker nor a native Postgres install. All verification — 70 automated tests plus the live demo in §7 — used SQLite. See the dedicated discussion below.

Concretely observed during this build, not theoretical:

- **Foreign-key enforcement is off by default in SQLite.** A test insert of a `fault_incident` row referencing a non-existent `equipment_id` succeeded with no error under the current SQLite setup (no `PRAGMA foreign_keys=ON` is issued). PostgreSQL enforces this by default, unconditionally.
- **Single-writer locking.** SQLite allows exactly one writer at a time on the whole file. This build already runs the worker and the dashboard as two concurrent processes against one file; at the spec's target scale (up to 100 dashboard users, each auto-refreshing every 5 seconds) this access pattern needs PostgreSQL's MVCC, where readers and writers don't block each other.
- **No native JSON/ENUM types in use.** The schema deliberately avoids PostgreSQL-specific types to stay SQLite-portable, which means the target API's richer `this_fault` per-component detail (see the integration spec) is not currently persisted anywhere durable — only the flat `fault_type` string is stored on each incident.
- **No Alembic migrations.** Tables are created via `Base.metadata.create_all()`. Fine for one evolving development schema; not appropriate for a production cutover with existing data.

5.3 Alerting

- **No real email has been sent.** The SMTP send path is unit-tested with `smtplib` mocked; no SMTP credentials were available in this environment to send an actual message.
- **No cross-cycle cooldown beyond incident-level dedup.** An incident is guaranteed exactly one "opened" alert and, on escalation, one further alert — but there is no separate configurable cooldown window (e.g. "don't alert on the same equipment more than once per hour regardless of incident boundaries").

5.4 Web / security

- **No CSRF token on mutating forms.** Login, subscription, and user-management POST endpoints rely on the session cookie's `SameSite=Lax` attribute (which already blocks the cookie from being sent on cross-site POST requests) rather than an explicit per-request CSRF token. This is meaningful protection, but not a complete substitute for one.
- **No rate limiting** on the login endpoint or any API route.
- **First admin requires shell access.** `db/seed_admin.py` must be run from a terminal — there is no in-app "first run setup" flow.
- **Fixed 7-day session lifetime**, not currently configurable via `.env`.

5.5 Operational

- **No retry-with-backoff within a single poll cycle.** A transient failure marks that cycle's `MonitoringRun` as `FAILED` and waits for the next scheduled poll, rather than the spec's suggested 3-attempt backoff within the same cycle.
- **No automatic recovery if the re-authentication browser itself crashes.**
- **Single-instance assumption.** Nothing in the design currently guards against two worker processes running concurrently against the same account (would cause duplicate polling and log noise, though the state-transition logic itself is idempotent enough not to create duplicate incidents).

6. Feature Walkthrough (Live Screenshots)

All screenshots below were captured with Playwright against the actual running application (a real `uvicorn` process, backed by a real poll loop) during this build — not recreated mockups. Three of the eight equipment rows visible (`[DEMO] Low Stock Unit` , `[DEMO] Faulty Unit`) are clearly-labeled synthetic additions, included specifically so every health-state badge could be demonstrated (see §5.1 on why no real fault has occurred yet). The other six rows — NEXUS, PNR, Gravity, REFRESH-1, REFRESHA 2, Warehouse — are the real fleet.

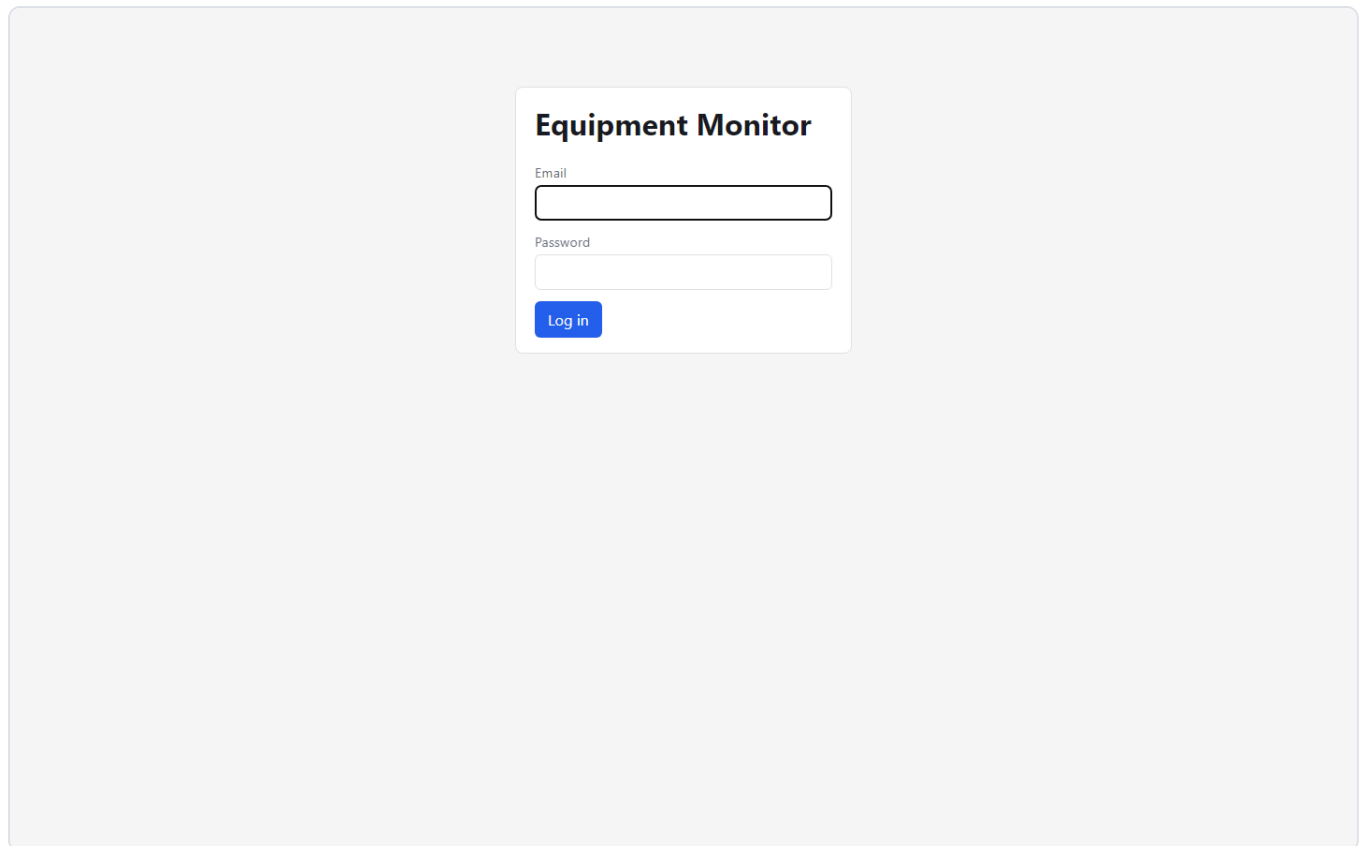
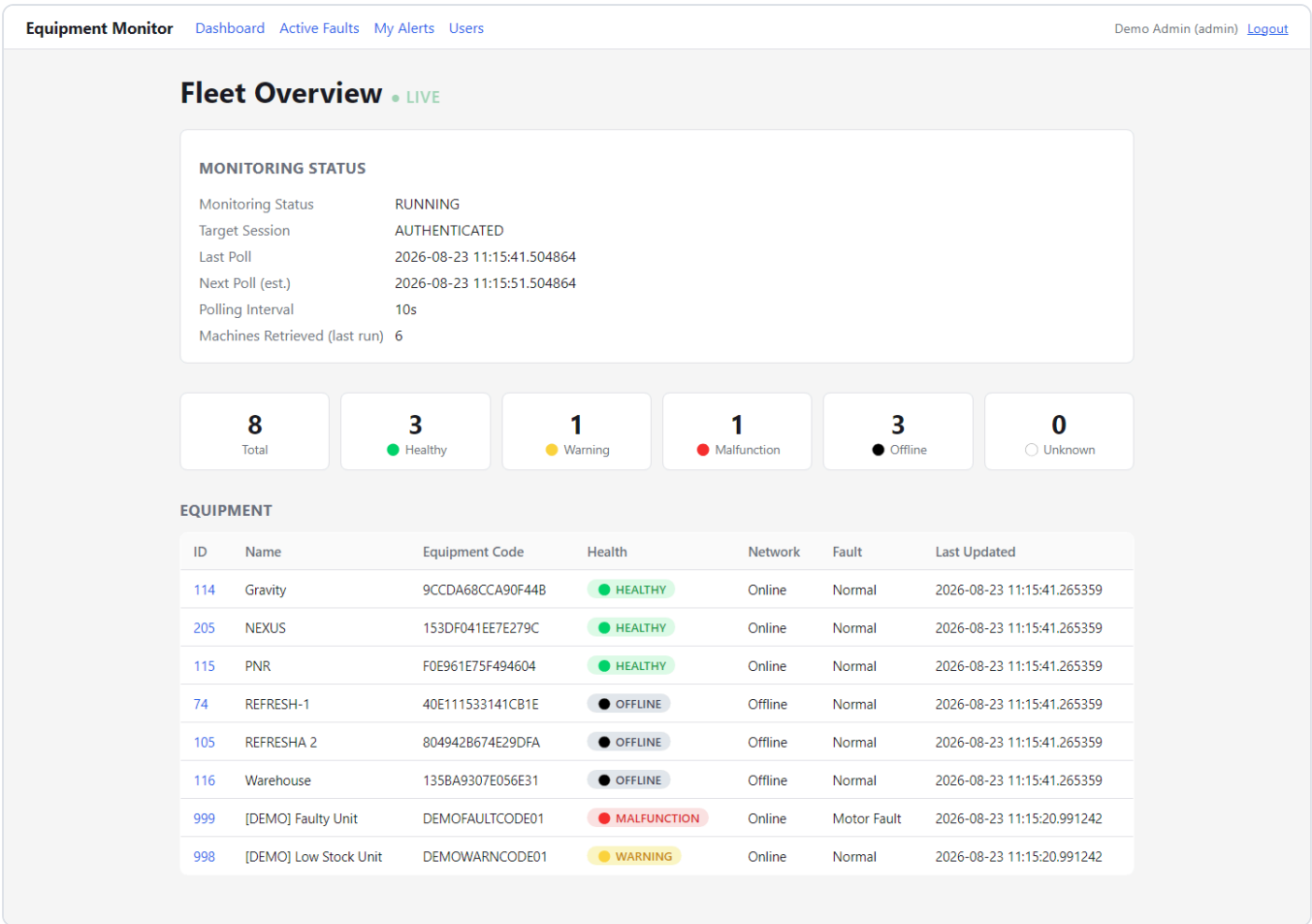


Figure 3 — Login page. Session-based authentication, distinct from the target application's own credentials.



EQUIPMENT

ID	Name	Equipment Code	Health	Network	Fault	Last Updated
114	Gravity	9CCDA68CCA90F44B	HEALTHY	Online	Normal	2026-08-23 11:15:41.265359
205	NEXUS	153DF041EE7E279C	HEALTHY	Online	Normal	2026-08-23 11:15:41.265359
115	PNR	F0E961E75F494604	HEALTHY	Online	Normal	2026-08-23 11:15:41.265359
74	REFRESH-1	40E111533141CB1E	OFFLINE	Offline	Normal	2026-08-23 11:15:41.265359
105	REFRESHA 2	804942B674E29DFA	OFFLINE	Offline	Normal	2026-08-23 11:15:41.265359
116	Warehouse	135BA9307E056E31	OFFLINE	Offline	Normal	2026-08-23 11:15:41.265359
999	[DEMO] Faulty Unit	DEMOFAULTCODE01	MALFUNCTION	Online	Motor Fault	2026-08-23 11:15:20.991242
998	[DEMO] Low Stock Unit	DEMOWARNCODE01	WARNING	Online	Normal	2026-08-23 11:15:20.991242

Figure 4 — Fleet overview: monitoring status panel, health-state counts, and the full equipment table with colored badges.

Note "Target Session: AUTHENTICATED" and a live poll interval of 10s — this is a running system, not a static page.

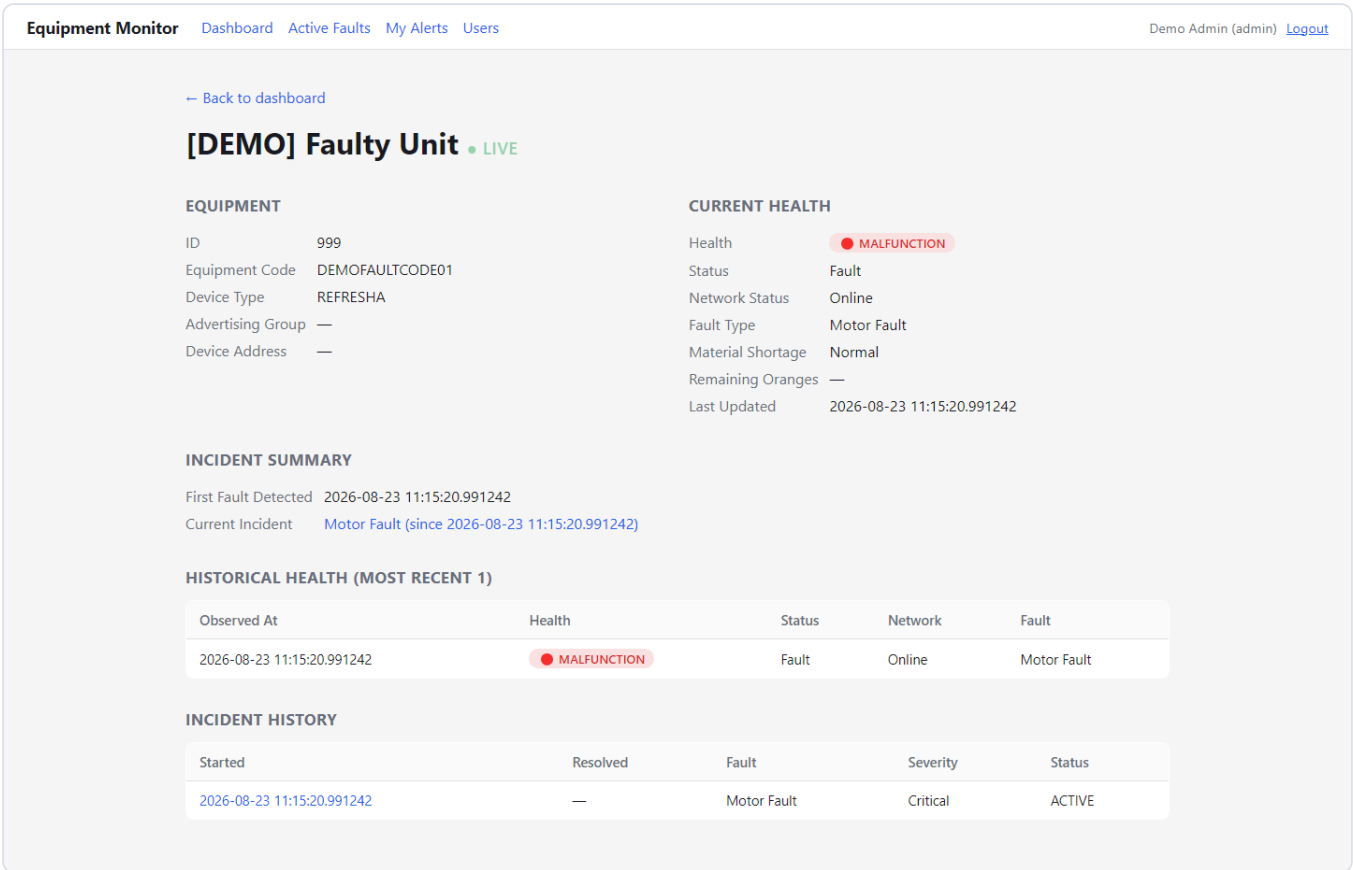


Figure 5 — Equipment detail: current health, incident summary, historical snapshot timeline, and full incident history for one device.

Equipment Monitor

[Dashboard](#)[Active Faults](#)[My Alerts](#)[Users](#)

Demo Admin (admin) [Logout](#)

Active Faults

LIVE

Equipment	Fault	Severity	Started	Duration
[DEMO] Low Stock Unit (998)	Normal	Warning	2026-08-23 11:15:20.991242	0:00:33.081813
[DEMO] Faulty Unit (999)	Motor Fault	Critical	2026-08-23 11:15:20.991242	0:00:33.081813
Warehouse (116)	Normal	Critical	2026-08-23 10:53:53.137453	0:22:00.935602
REFRESHA 2 (105)	Normal	Critical	2026-08-23 10:53:53.137453	0:22:00.935602
REFRESH-1 (74)	Normal	Critical	2026-08-23 10:53:53.137453	0:22:00.935602

Figure 6 — Active faults across the fleet, including the three genuinely offline devices (real data) and the two synthetic demo rows (Warning/Critical).

Equipment Monitor

[Dashboard](#)[Active Faults](#)[My Alerts](#)[Users](#)

Demo Admin (admin) [Logout](#)

[← Back to active faults](#)

Fault Details

Equipment

[\[DEMO\] Low Stock Unit](#)

Equipment ID

998

Equipment Code

DEMOWARNCODE01

Fault

Normal

Severity

Warning

Detected

2026-08-23 11:15:20.991242

Resolved

—

Duration

0:00:33.864640

Current State

ACTIVE

Notification Sent

No

Figure 7 — Individual incident detail: equipment, exact preserved fault text, severity, detection time, and notification status.

Equipment Monitor

[Dashboard](#)[Active Faults](#)[My Alerts](#)[Users](#)

Demo Admin (admin) [Logout](#)

My Alert Subscriptions

Machine	Severity	Channel	Enabled
No subscriptions yet — add one below.			

ADD SUBSCRIPTION

Machine

All machines

Severity

All severities

Add Subscription

Figure 8 — Per-user alert subscription management: subscribe to specific equipment or all machines, filtered by severity.

Equipment Monitor

[Dashboard](#)[Active Faults](#)[My Alerts](#)[Users](#)

Demo Admin (admin) [Logout](#)

Users

Name	Email	Role	Active
Demo Admin	demo@example.com	admin	Yes Deactivate

ADD USER

Name

Email

Password

Role

User

Add User

Figure 9 — Admin-only user management. Password hashes are never rendered here or anywhere else in the UI.

7. Testing & Live Verification Summary

7.1 Automated tests

Test file	Count	Covers
<code>test_health_engine.py</code>	11	Rule table: each spec example, priority order, case-insensitivity, UNKNOWN fallback
<code>test_state_manager.py</code>	10	Incident open/escalate/resolve; reproduces the spec's own "4 faults, 1 alert" worked example exactly
<code>test_notification_service.py</code>	6	SMTP interaction (mocked), console fallback, credentials never logged
<code>test_alert_engine.py</code>	12	Recipient resolution matrix: admin/subscription/severity/equipment-scoping/dedup
<code>test_lightweight_client.py</code>	14	Cookie loading, redirect-to-login detection, pagination, error classification — via <code>httpx.MockTransport</code> , no real network
<code>test_backend.py</code>	17	Auth, RBAC, all dashboard pages, subscription CRUD — via FastAPI's <code>TestClient</code>
Total	70	All passing at time of writing

7.2 Live verification performed

- **Real target connectivity (TC-001).** A numbered, documented live test case against the actual target site — authenticate (or reuse session), retrieve all equipment, report current status. Passing.
- **Session reuse across time.** Confirmed on a second run, hours after the first, that the saved session was still valid and no re-login occurred.
- **Browser elimination.** Ran the continuous poll loop against the real target and checked the OS process list (`Get-CimInstance Win32_Process`) across multiple full cycles — zero Playwright-owned Chromium processes existed at any point, confirming the hybrid lightweight-client design works as intended in practice, not just in theory.
- **Real incident lifecycle.** During live runs, the worker correctly opened incidents for genuinely offline equipment (Warehouse, REFRESHA 2, REFRESH-1) and did not duplicate them on subsequent polls.
- **Live dashboard.** A real `uvicorn` process was hit with real `curl` HTTP requests (not the in-process test client) — login, RBAC-gated pages, and a "Last Poll" timestamp confirmed to advance between two requests 12 seconds apart, proving end-to-end freshness rather than a cached/static response.

7.3 Not yet verified

Item	Reason
Real PostgreSQL	No Docker/Postgres available in this build environment
Real SMTP delivery	No SMTP credentials available in this build environment
DOM-scrape fallback under a real API failure	The JSON API path has not failed during this build
Behavior under a genuine target-side fault	No real fault has occurred on the connected account

8. Pending Features / Roadmap

8.1 Immediate — closing verification gaps

- Run the full stack against real PostgreSQL (`docker compose up -d && python -m db.init_db`) once Docker/Postgres is available.
- Configure real SMTP credentials and confirm an actual email is delivered end to end.

8.2 Phase 6 — Production hardening (not started)

- Alembic migrations, replacing `Base.metadata.create_all()` .
- Retry-with-backoff within a single poll cycle (3 attempts, 5s/10s waits, per the original spec) rather than deferring to the next scheduled poll on any transient failure.
- Browser crash/restart recovery for the re-authentication path.
- Structured logging, a health-check endpoint, and Prometheus/Grafana integration.
- Docker images for the worker and backend processes, plus an Nginx reverse proxy.
- A secrets manager in place of plain `.env` files.
- Database backups and a documented restore procedure.
- Rate limiting on the login endpoint and API routes.
- A CSRF token on mutating forms, on top of the existing `SameSite=Lax` protection.
- An in-app first-admin setup flow, replacing the current CLI-only `seed_admin.py` .

8.3 Feature gaps

- **Multiple monitoring profiles** (more than one target account), as described in the original spec's scale section.
- **Rich per-component fault detail.** The target API's `this_fault` field (individual motor/sensor-level fault flags) is a strong candidate for a JSONB column once on PostgreSQL — currently unused beyond the flat `fault_type` string.
- **Confirmed real-world health-rule vocabulary** — the current rule table is correct against the spec's illustrative examples but has not been validated against an actual observed fault from this target account.
- **Configurable session lifetime** for dashboard logins (currently a fixed 7 days).
- **Optional Next.js frontend**, layered on the existing `backend/api/` routes, if still wanted after using the server-rendered version.
- **A per-equipment/severity notification cooldown window**, independent of incident boundaries.

Appendix A — Tool & Technology Reference

Tool	Version	Category	Key properties / why used here
Python	3.10	Runtime	Async support, broad library ecosystem, single language across worker + backend.
Playwright	1.47.0	Browser automation	Cross-browser automation library; used here exclusively for the CAPTCHA login flow and as a DOM-scrape fallback — not for steady-state polling.
httpx	0.27.2	HTTP client	Async-native HTTP client with a cookie jar and redirect handling; powers all steady-state polling (no browser) and FastAPI's test client.
FastAPI	0.115.0	Web framework	Async, type-hint-driven request validation, automatic OpenAPI docs, dependency-injection system used for auth/RBAC/DB session wiring.
Uvicorn	0.31.0	ASGI server	Serves the FastAPI app; [standard] extras include <code>uvloop</code> / <code>httptools</code> for performance.
Jinja2	3.1.4	Templating	Server-side HTML rendering for the dashboard — no client build step required.
python-multipart	0.0.12	Web	Required by FastAPI/Starlette to parse HTML form POST bodies (login, user/subscription forms).
SQLAlchemy	2.0.35	ORM	Declarative models + Core; the same model definitions run against both PostgreSQL and SQLite by using only portable column types.
psycopg2-binary	2.9.9	DB driver	Synchronous PostgreSQL driver used by SQLAlchemy for the production database target.
PostgreSQL	16 (docker image)	Database	Specified production RDBMS. ACID, MVCC (non-blocking concurrent readers/writers), enforced foreign keys by default, native JSON/ENUM types.
SQLite	bundled (Python stdlib)	Database	Embedded, file-based; used for the automated test suite and the local/demo run described in this report. Not the production target — see §5.2.
PyYAML	6.0.2	Config	Loads <code>config/health_rules.yaml</code> — keeps health-evaluation rules editable without a code change.
python-dotenv	1.0.1	Config	Loads <code>.env</code> into process environment variables; keeps all secrets out of source code.
pytest	8.3.3	Testing	Test runner for all 70 unit/integration tests.

Tool	Version	Category	Key properties / why used here
pytest-asyncio	0.24.0	Testing	Enables <code>async def</code> test functions (used by the lightweight-client tests).
hashlib / hmac / secrets (stdlib)	—	Security	PBKDF2-HMAC-SHA256 password hashing and HMAC-signed session cookies — no external auth dependency.
smtplib / email (stdlib)	—	Alerting	SMTP email delivery — no external mail library required.
Docker / docker-compose	—	Infrastructure	Containerized local PostgreSQL for development, as specified in <code>docker-compose.yml</code> .

Appendix B — Project File Structure

discovery/inspect.py	One-off manual DOM/network capture tool
docs/	Integration spec, this report
monitoring/	
config.py	env-driven settings (DB, SMTP, web, target)
models.py	MonitoringState, EquipmentRecord, exceptions
selectors.py	target URLs / API paths / field maps
mapping.py	raw-row → canonical-field mapping (shared)
lightweight_client.py	httpx-based client – steady-state polling
browser_manager.py	Playwright browser/context lifecycle
session_manager.py	session-aware auth (reuse-first)
target_client.py	Playwright client (login/CAPTCHA + fallback)
equipment_extractor.py	canonical rows → validated EquipmentRecord
validator.py	zero-records guard
worker.py	real poll-cycle entry point
poc_runner.py	Phase 2 connectivity demo
db/	
models.py	equipment, snapshots, incidents, users, ...
base.py	engine/session (incl. DATABASE_URL override)
init_db.py, seed_admin.py	setup CLIs
config/health_rules.yaml	configurable health-evaluation rules
services/	
health_engine.py	raw fields → canonical HealthState
state_manager.py	incident dedup/escalation/resolution
notification_service.py	pluggable delivery channel
alert_engine.py	recipient resolution + message content
backend/	FastAPI dashboard
main.py, deps.py, security.py, templating.py	
api/ auth.py, equipment.py, alerts.py, users.py, monitoring.py	
templates/, static/	Jinja2 HTML, one stylesheet
tests/	70 automated tests + live test cases

Appendix C — State Vocabulary Glossary

Two entirely separate vocabularies are used throughout this application, and keeping them distinct is a core design rule (never let a technical failure masquerade as equipment health):

Equipment health states (what a machine is doing)

State	Meaning
HEALTHY	Status/network/fault/material all nominal.
WARNING	Material shortage present, no active fault.
MALFUNCTION	Fault type reported and not recognized as "no fault".
OFFLINE	Network connection reported offline.
UNKNOWN	A required field was missing or an unrecognized value was reported — deliberately <i>not</i> treated as a fault (requirement #28).

Monitoring/technical states (what the monitoring system is doing)

State	Meaning
AUTHENTICATION_REQUIRED	Session invalid/expired and re-authentication could not complete.
TARGET_UNAVAILABLE	The target application could not be reached.
EXTRACTION_ERROR	Data was reachable but failed to parse/validate.
MONITORING_ERROR	An unclassified failure in the monitoring pipeline itself.

None of the four technical states above ever writes to an equipment's health state — they are recorded only on that cycle's `MonitoringRun` row.